Gemeinsame Schlüssel

Python / PM4Py 2.7.23.11 | tatsächliche Werte | Rolle und Begründung getrennt

Parameter	Wert	Bedeutung	Begründung / Grenze
activity_key	concept:name	Aktivitätslabel für alle Miner.	Originale Namen einschließlich Leerzeichen bleiben erhalten.
timestamp_key	time:timestamp	Zeitstempelattribut; beim Genetic Miner auch für interne Sortierung verwendet.	Alle Fallsequenzen nach nativer Sortierung gegen dieselbe Completion-Projektion geprüft.
case_id_key	case:concept:name	DataFrame-Schlüssel; im EventLog liegen IDs als Trace-Attribut concept:name vor.	Alle 31,509 Fall-IDs und Fallgewichte bleiben erhalten.

Priorität: Soundness + ungefähr 80 % exakte Fallabdeckung; danach Precision erhöhen.
Harte Guards: aktuell 10,19 % vollständige Testfallabdeckung; Struktur ohne Guards: 80,45 %.
Metriken stets auf derselben Eingabe und mit ihren Ausschlüssen vergleichen.

Details und Quellen: docs/MINER_PARAMETERS.md | Begründungen: docs/DESIGN_DECISIONS.md

Inductive Miner und IMf

Python / PM4Py 2.7.23.11 | tatsächliche Werte | Rolle und Begründung getrennt

Parameter	Wert	Bedeutung	Begründung / Grenze
noise_threshold	0.0	Relative Rauschschwelle; 0 aktiviert IM, positive Werte IMf. Kein Anteil entfernter Fälle.	0 als Vergleichsbasis ohne zusätzlichen Rauschfilter; im finalen Modell 0.03 nach dokumentierter Kandidatensuche.
multi_processing	false	Parallele Ausführung des Miners. Kein fachlicher Filter.	False begrenzt Prozess-/Speicheraufwand und vereinfacht Fehlersuche. Kein Qualitätsoptimierungshebel.
disable_fallthroughs	false	False erlaubt zusätzliche Rückfallstrategien, wenn kein strukturierter Cut gefunden wird.	Standardstrategien bleiben verfügbar. True lässt hier weiterhin Empty-Trace-/Flower-Fallback zu; es garantiert keine höhere Präzision.
final noise_threshold	0.03	Rauschschwelle des gewählten IMf-Kandidaten, bei anderer Discovery-Trainingsmenge.	0.03 wurde in der Suche ausgewählt; kein Beweis einer generell optimalen 3-Prozent-Schwelle.
training_coverage	0.6	Externer Frequenzfilter für das finale Modell; kein interner IM-Parameter.	Häufige Varianten als Discovery-Fokus. Tatsächlich 18,918 Fälle und 73 Varianten; Bewertung weiterhin auf allen 31,509 Fällen.
target coverage band	[78, 82]	Eigenes Auswahlband für ungefähr 80 Prozent vollständig replaybare Fälle.	Operationalisierung der Aufgabenanforderung; keine Datenbereinigung und kein vom Datensatz vorgegebener Grenzwert.

Wichtig: 0.03 ist eine relative Rauschschwelle, kein Anteil gelöschter Fälle.
Das finale Modell verwendet zusätzlich eine häufigkeitsbasierte Discovery-Trainingsmenge.
Mehr Rauschfilterung garantiert keine monoton höhere Precision.

Details und Quellen: docs/MINER_PARAMETERS.md | Begründungen: docs/DESIGN_DECISIONS.md

Heuristics Miner CLASSIC

Python / PM4Py 2.7.23.11 | tatsächliche Werte | Rolle und Begründung getrennt

Parameter	Wert	Bedeutung	Begründung / Grenze
variant	CLASSIC	Verwendete Heuristics-Miner-Variante; Ausgabe wird in ein Petri-Netz umgewandelt.	Festgelegte Vergleichsbasis. Ein exportiertes Workflow-Netz ist noch kein Soundness-Nachweis.
dependency_thresh	0.5	Mindest-Abhängigkeitsmaß für eine direkt folgende Relation.	0.5 ist die dokumentierte Baseline, nicht datenoptimiert. Höher ist strenger; seltene reale Pfade können entfallen.
and_measure_thresh	0.65	Schwelle für die Interpretation benachbarter Relationen als Parallelität.	0.65 als Bibliotheksstandard. Höhere Werte erschweren AND-Erkennung; keine globale Monotonie der Qualitätswerte.
loop_length_two_thresh	0.5	Schwelle zum Erkennen kurzer Schleifen wie A-B-A.	0.5 als Standard; kurze Wiederholungen sollen nicht durch reine Richtungsabhängigkeit verloren gehen.
min_act_count	1	Absolute Mindestanzahl beobachteter Events pro Aktivität.	1 vermeidet eine zusätzliche absolute Häufigkeitsselektion. Andere HM-Filter gelten trotzdem.
min_dfg_occurrences	1	Absolute Mindesthäufigkeit einer direkt folgenden Kante.	1 als permissive absolute Baseline; Dependency und relatives Pre-Cleaning können Kanten weiterhin entfernen.
dfg_pre_cleaning_noise_thresh	0.05	Relative DFG-Bereinigung vor der Abhängigkeitsberechnung.	0.05 als expliziter Standard. Bezieht sich auf lokale Kantenhäufigkeiten, nicht auf fünf Prozent gelöschte Fälle.

Die Baselinewerte sind dokumentierte Standardwerte, keine Optimierung auf diesen Datensatz.
Das aktuelle HM-Netz ist nachweislich unbeschränkt. Hohe Precision allein genügt nicht.
Die Folgen dieser Wahl sind Teil des Ergebnisvergleichs.

Details und Quellen: docs/MINER_PARAMETERS.md | Begründungen: docs/DESIGN_DECISIONS.md

Genetic Miner CLASSIC: dritter Versuch

Python / PM4Py 2.7.23.11 | tatsächliche Werte | Rolle und Begründung getrennt

Parameter	Wert	Bedeutung	Begründung / Grenze
population_size	10	Anzahl konkurrierender Modelle je Generation.	10 entspricht dem offiziellen kleinen Beispiel und hält den Voll-Log-Versuch bezahlbar; Default 500 wäre deutlich teurer.
generations	6	Obergrenze einschließlich der Anfangspopulation; native Frühstopps sind möglich.	6 gibt vorab maximal 60 Modellbewertungen in bis zu 6 Populationen vor. Explorationsbudget, keine Konvergenzgarantie.
elitism_rate	0.1	Anteil unverändert übernommener bester Individuen; Anzahl ist round(rate * population_size).	0.1 erhält bei Population 10 genau ein Modell. Default 0.01 ergäbe bei diesem kleinen Budget null Eliten.
crossover_rate	1.0	Wahrscheinlichkeit zur Rekombination ausgewählter Eltern.	1.0 übernimmt den nativen Standard und kombiniert Eltern bei jeder entsprechenden Auswahl.
mutation_rate	0.01	Wahrscheinlichkeit je Input-/Outputeintrag, vorhandene Nachbarn neu zu partitionieren.	0.01 übernimmt den Standard; dies ist nicht ein Prozent beliebig neu eingefügter Kanten.
elitism_min_sample	2	Tatsächlich die Stichprobengröße der Eltern-Turnierauswahl; nicht die Zahl der Eliten.	2 hält bei Population 10 den Selektionsdruck moderat und ermöglicht mehr Elternvielfalt als der Default 5.
random / numpy seed	20261025	Kontrolliert Python-random und NumPy-Zufall.	Fixiert den konkreten Versuch. Ein Seed belegt keine Robustheit über mehrere Suchläufe.
PYTHONHASHSEED	20261025	Fixiert Set-Reihenfolgen im separaten Python-Prozess.	Zusätzlich nötig, weil die Implementierung zufällig aus Set-basierten Pools zieht.
log_csv	evolution.csv	Protokolliert die nativen Fitnesswerte je Population.	Unterscheidet geplante und tatsächlich durchgeführte Generationen sowie das interne Ziel von externen Qualitätsmetriken.

Native Zielfunktion: 0.4 × mittlere Trace-Fitness + 0.6 × Anteil passender Traces.
Precision wird intern nicht optimiert. Maximal 60 Modellbewertungen in bis zu 6 Populationen; tatsächliche Generationen im Laufprotokoll.
Tatsächlich 2 Generationen; Modell unbeschränkt. Explorationslauf, keine globale Optimalitätsbehauptung.

Details und Quellen: docs/MINER_PARAMETERS.md | Begründungen: docs/DESIGN_DECISIONS.md